

## Video Summary

This Telusko lesson compares several ways to swap the values of two variables: using a temporary variable, arithmetic operations, bitwise XOR, and the Pythonic tuple-unpacking technique.

Source video: <https://www.youtube.com/watch?v=3dpJrMtxYeo>

### 1

## What Swapping Means

Swapping exchanges two existing values without losing either value. If a starts as 5 and b starts as 6, after the swap a should be 6 and b should be 5.

**a,b****Start**

a stores 5 and b stores 6 before the swap begins.

**Swap**

The method must protect both original values during assignment.

**6,5****End**

a should become 6 and b should become 5.

```
>>> a = 5
>>> b = 6
>>> a, b = b, a
>>> a, b
(6, 5)
```

swap goal

## 2

## Use a Temporary Variable

The traditional method stores one value in a third variable before overwriting it. It is beginner-friendly and works across many programming languages.

| Step | Statement             | Reason                     |
|------|-----------------------|----------------------------|
| 1    | <code>temp = a</code> | Save original a.           |
| 2    | <code>a = b</code>    | Move b into a.             |
| 3    | <code>b = temp</code> | Restore original a into b. |



```
a = 5
b = 6

temp = a
a = b
b = temp

print(a)
print(b)

Output:
a = 6
b = 5
```

### Trade-off

This method is simple and safe, but it uses one extra variable as temporary storage.

## 3

## Use Arithmetic Operations

Addition and subtraction can swap two numeric values without a third variable. The technique saves extra storage, but it performs more operations and is less readable.

| Formula     | What it stores        | Example value          |
|-------------|-----------------------|------------------------|
| $a = a + b$ | Total of both values. | $5 + 6 \rightarrow 11$ |
| $b = a - b$ | Original a.           | $11 - 6 \rightarrow 5$ |
| $a = a - b$ | Original b.           | $11 - 5 \rightarrow 6$ |

```
addition-subtraction swap

a = 5
b = 6

a = a + b
b = a - b
a = a - b

print(a, b)

Output:
6 5
```

### Design trade-off

Arithmetic swapping avoids a third variable, but it is limited to numbers and adds CPU work. In Python, readability usually matters more.

## 4

## Use Bitwise XOR

XOR is an advanced integer-only method. It swaps the bit patterns directly, avoiding an explicit temporary variable and avoiding arithmetic overflow concerns in lower-level languages.

^

### Integer only

XOR works on integer bit patterns, not strings or general objects.

xor

### No temp name

The values are transformed through repeated XOR assignments.

!

### Less readable

It is useful to know, but not the normal Python choice.

```
bitwise XOR swap

a = 5
b = 6

a = a ^ b
b = a ^ b
a = a ^ b

print(a, b)

Output:
6 5
```

### Why XOR appears here

XOR is shown because it is a classic interview-style swap. For normal Python programs, tuple unpacking is clearer.

## 5

## Use the Pythonic Way

The recommended Python approach is one line: `a, b = b, a`. It is short, readable, and works because the right side is evaluated before the left side receives new values.



```
a = 5
b = 6

a, b = b, a

print(a)
print(b)

Output:
a = 6
b = 5
```

| Stage           | Python action                                 | Result       |
|-----------------|-----------------------------------------------|--------------|
| Tuple packing   | Fetch b and a on the right side.              | (6, 5)       |
| Tuple unpacking | Assign first item to a and second item to b.  | a=6, b=5     |
| One statement   | Both targets receive the new values together. | No data loss |

### Key idea

Python evaluates the right-hand side first, then unpacks the produced pair into the left-hand side variables.

## 6

## Choose the Right Method

All four methods teach something useful, but the Pythonic method should be your default in real Python code.

| Method             | Extra storage   | Works best for     | Use in Python      |
|--------------------|-----------------|--------------------|--------------------|
| Temporary variable | Uses temp name. | Any value type.    | Good for learning. |
| Arithmetic         | No temp name.   | Numbers only.      | Rarely needed.     |
| XOR                | No temp name.   | Integers only.     | Advanced demo.     |
| Tuple unpacking    | Implicit pair.  | Most Python swaps. | Recommended.       |

```
# Practice: compare all methods with a = 5, b = 6

a, b = 5, 6
a, b = b, a
print("Pythonic:", a, b)

# Expected output:
Pythonic: 6 5
```

### Day 18/55 takeaway

Swapping is about preserving both values while exchanging them. Learn the traditional, arithmetic, and XOR techniques, but prefer `a, b = b, a` in Python.